

Agent-based Modeling of Stock Market Prices

MAT 586

June 13, 2018

Nolan Gagnon

University of Maine

Abstract: In this project, the effects of various trading strategies on stock market prices are studied. Agent-based modeling (where the agents are traders with different strategies) is used in a simulation of price fluctuations for six different stocks from the American stock markets. In this small simulation, it was observed that high-frequency trading strategies can lead to more dramatic price swings under certain market conditions.

Introduction

Active participants in the stock markets (i.e., those who make their own trades) have various strategies for trying to make money. Those with little knowledge of economics and finance tend to make decisions based either on trends they see in stock charts or on what highly regarded market analysts have to say. If a particular stock has been making large price jumps in the positive direction and there is a lot of hype about the underlying company (think Tesla, Inc. or Nvidia Corp.), these sorts of investors will jump on board and buy as many shares as they can afford. When the situation is reversed (i.e., the stocks are making large price jumps in the negative direction and bad news about the underlying company has been released), these same traders will sell their shares as fast as they can. Market participants who use this strategy are called *momentum traders*.

Then there are the more intelligent investors who base their trades on *fundamental analysis*. Fundamental analysis is the process of making a valuation of a company based on the financial documents it is required to file with the Securities and Exchange Commission (SEC) and on current (as well as predicted) economic conditions. Warren Buffett is the best example of a fundamental trader. He only buys shares in a company that he believes is undervalued (i.e., worth more than what the stock market currently values it at). An example of a fundamental-analysis-based trade is Buffett's decision to invest in Bank of America Corporation (BAC) in 2011 in the aftermath of the Financial Crisis when BAC shares were selling for around \$7.00 each. His analysis of the company led him to conclude that it was worth far more than that on a per-share basis (BAC shares now sell for around \$30.00 each, so he was correct).

There is another major family of trading strategies known as quantitative trading (quant trading), which requires knowledge of mathematics, statistics, computer engineering, electrical engineering, and physics in addition to knowledge of economics and finance. Quant traders develop mathematical and statistical models of the assets they intend to trade and write computer programs to make trades based on their models algorithmically. In recent years, with the vast improvements in computing power, a type of quant trading known as *high-frequency trading* (HFT) has become popular. HFT funds are capable of making high-volume trades in milliseconds based on minuscule price fluctuations that normal traders cannot take advantage of.

HFT funds have their critics, though. Some experts believe that HFT causes more dramatic price swings. In 2010, there was an event called the Flash Crash, where approximately \$1 trillion were lost (and then regained) in U.S. stock markets over the course of 36 minutes. There are many theories as to what caused this crash, one of which blames HFT algorithms. This is the inspiration for this project.

Considering supply and demand dynamics, it does seem reasonable to think that HFT could cause crashes like the Flash Crash of 2010. If bad news about a company is released and HFT algorithms react by selling millions of shares, the supply is going to increase rapidly. At the same time, demand is going to decrease due to panicking non-fundamental traders. It is certainly feasible that this decreasing demand and supply spike could cause a stock's price to drop significantly and rapidly.

Model/Methods

In the simulation, there are four different types of traders buying and selling stocks to each other. As they do this, supply and demand fluctuate, causing stock prices to fluctuate as well. The simulation is written in the statistical computing language R. It consists of a main function that runs the simulation inside a *for* loop, as well as some initialization and helping functions to make the code more organized.

There are three initialization functions that run before the price fluctuation simulation begins. One downloads (from Yahoo! finance) the current prices of the stocks used in the simulation. This allows the simulation to always start out with real stock prices. The other two initialization functions distribute wealth and shares, respectively, among the traders. Both wealth and shares are distributed according to a Pareto distribution to make the simulation as realistic as possible. This means that a small percentage of the traders own most of the shares and wealth (a vulgar way of putting it is that “the rich get richer, and the poor get poorer”).

Most of the remaining functions are used to execute the various trading strategies and to stimulate the market. An example of market stimulation is the release of a company’s Form 10-Q (quarterly performance report). In the simulation, each company has a financial health vector with four values. The values are the probabilities that the company issues a very negative 10-Q, a negative 10-Q, a positive 10-Q, and a very positive 10-Q, respectively. As an example, a relatively healthy company like Skyworks Solutions, Inc. (SWKS) (which is currently debt free) would have high probabilities of issuing positive or very positive Form 10-Qs. A terrible company like Blue Apron Inc. (APRN) (whose stock price plummeted after the initial public offering last year) would have high probabilities of issuing negative or very negative Form 10-Qs.

There is one other function that deserves special mention. It is a function that produces stock charts after the simulation completes. Stocks whose price increased during the simulation are shown in green and stocks whose price decreased are shown in red.

The simulation parameters are the number of time steps to run the simulation for, the number of momentum traders, number of fundamental traders, number of HFT traders, number of random traders, and the price movement tick size. A tick size of 0.001 is used by default because it makes the stock charts look the most realistic. Additionally, one can set the frequency at which the HFT traders make decisions using the *hft_freq* parameter. For example, if *hft_freq* = 10, then HFT traders will make decisions every 10 time steps. One should note, however, that the lower this parameter is, the more slowly the simulation will run.

The state variables are the stock prices, the amount of wealth a trader has, the number of shares a trader has, and the levels of supply and demand. These all fluctuate during the simulation. For organizational purposes, all of these are stored in data frames.

The exact trading strategies used in the current version of the model are relatively simplistic (but will be made more complex in future versions). Momentum traders trade stochastically, but are more likely to buy a stock if its current price exceeds the n -step moving average by at least 2% (n can be set by the user at the beginning of the simulation), and they are more likely to sell a stock if the current price is below the n -step moving average by at least 2%. It should also be noted that they only trade every n steps.

Fundamental traders only make trades every quarter (meaning they trade at the lowest frequency, in most cases). Their decisions are based on quarterly earnings reports, which can either be VERY GOOD, GOOD, BAD, or VERY BAD. If a company issues a VERY GOOD earnings report, fundamental traders are more likely to buy the company's stock. If, on the other hand, a company issues a VERY BAD earnings report, fundamental traders are more likely to sell the company's stock. Finally, HFT traders make decisions based on earnings reports and n -step moving averages, but they make their decisions much faster than the other traders (unless a very large number is used for hft_freq).

Whenever a trader buys a stock, he buys 100 shares if he has enough capital to do so. Similarly, whenever a trader sells a stock, he sells 100 shares if he has at least 100 shares. Demand goes up by 10 units when 100 shares are bought by any trader and supply goes down by 10 units. Similarly, supply goes up by 10 units when 100 shares are sold and demand goes down by 10 units. This was done to keep prices from getting unrealistically high.

Analysis/Results

In Figure 1 below, we see the price simulation results when HFT traders trade at the same speed as momentum traders (every 40 time steps). A total of 300 time steps were used to obtain this figure. In

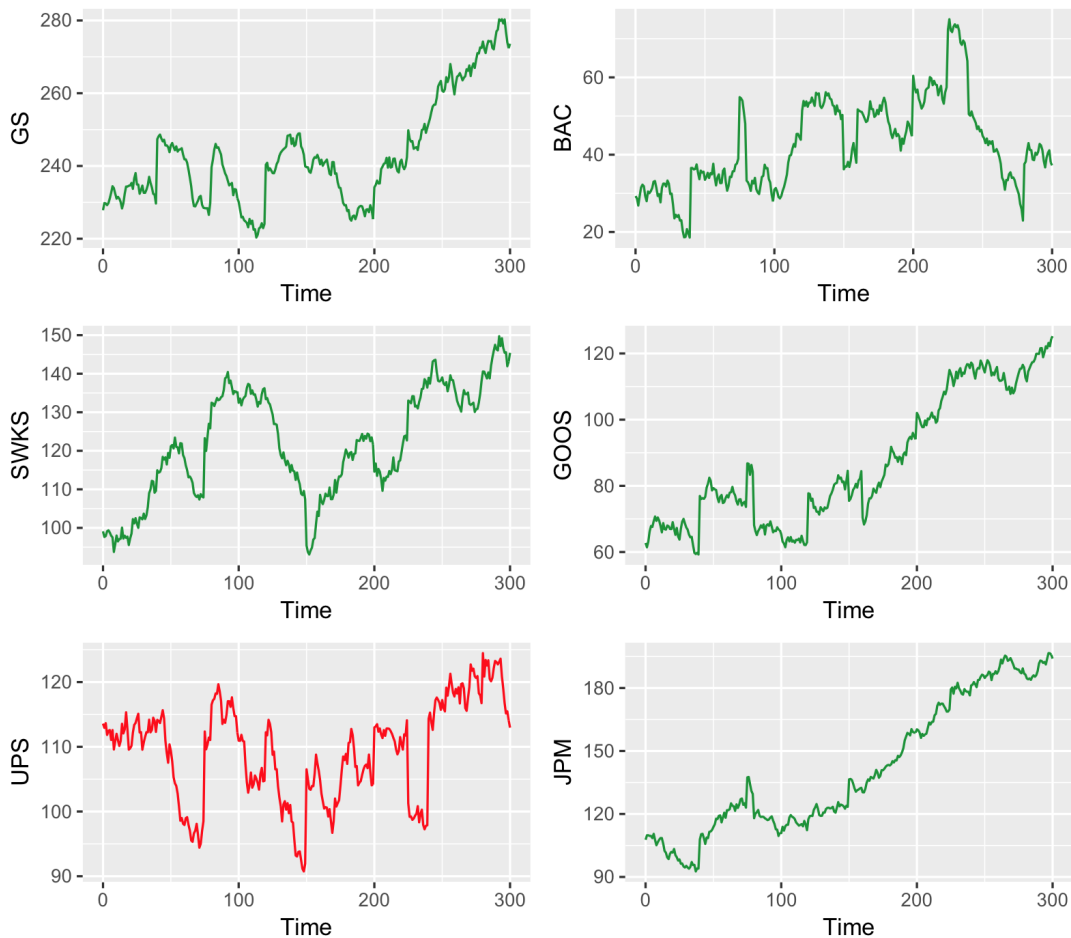


Figure 1: Historical prices when HFT traders trade at the same speed as momentum traders

most of the charts, we can see significant jumps occurring at times 75, 150, and 225. All of the stock prices

went up overall during this simulation, except for the price of UPS stock. But all the stocks (except for JPM) experienced noticeable volatility along the way. Now observe Figure 2 below.

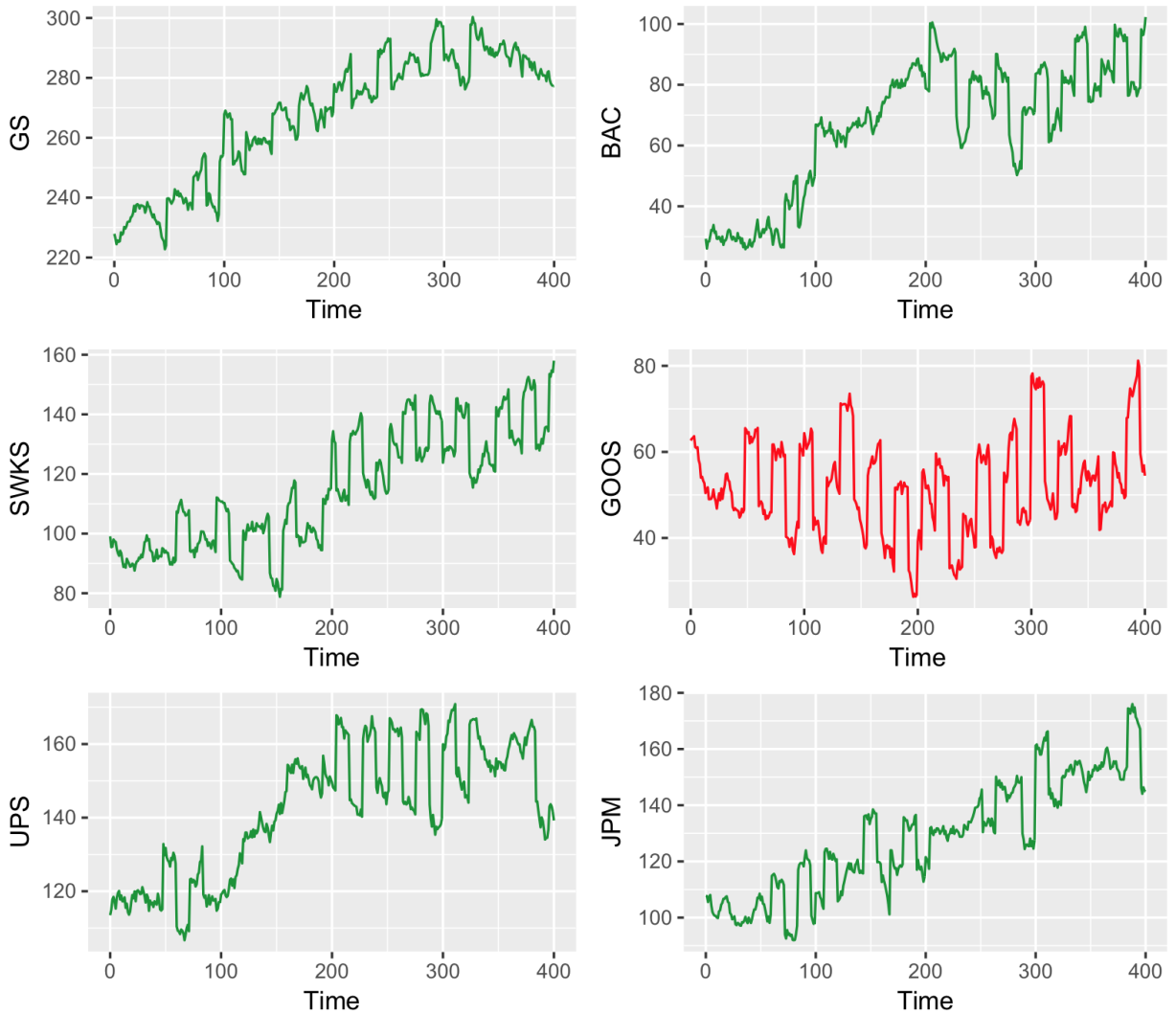


Figure 2: Historical prices when HFT traders trade four times faster than momentum traders

This figure was produced using 400 time steps. Momentum traders traded every 40 steps, and HFT traders traded every 10 steps ($hft_freq = 10$, $mavg_l = 40$). Most charts have significant jumps at times 100, 200, and 300. But there are significant jumps at other times as well, especially for GOOS. Volatility overall has increased significantly.

Discussion

The price jumps that occurred at times 75, 150, and 225 during the first simulation from the previous section were most likely the effect of the fundamental traders. With 300 total time steps, times 75, 150, and 225 would have been the quarters. Since the HFT traders were trading at the same rate as the momentum traders, there were not a lot of wild price swings in the stock charts.

In the second simulation, each stock (especially GOOS) exhibited steeper price jumps. This is most likely due to the HFT traders trading several times faster than anyone else. Even though the model

is currently rather simplistic and doesn't include several important market forces that exist in the real markets, the results of the simulation do seem to provide evidence for the notion that HFT trading can cause more dramatic price swings.

In both simulations, all but one stock went up in value overall. This is most likely due to the fact that the companies used in the simulation were relatively healthy (in my opinion), and their financial health vectors reflected that.

For future work, several aspects of the model need to be changed. First and foremost, the code itself needs to be rewritten. The trading strategies were not executed in a vectorized fashion, making the simulation run slowly. R is good for building and testing a model quickly, but it is too slow for our purposes. In the future, the computing-intensive portions of the code will be rewritten in C and parallelized. The output data will then be passed to R or MATLAB for visualization purposes.

The code also does not have error checking and handling at the moment. The web-scraping part of the code that downloads stock prices from the Yahoo! Finance sometimes gets the wrong piece of data (or fails to connect to the Internet). This type of error needs to be handled.

There were several market forces that were not applied to this model. One of the most important ones that needs to be added to future models is the Federal Reserve's regulation of the economy. The Federal Reserve changing its monetary policy can have a dramatic impact on the stock market. For example, when they raise interest rates, traders tend to take money out of the stock market and put it into bonds instead (because they are safer). The opposite happens when the Federal Reserve lowers interest rates. Traders moving money in and out of the stock market because of Federal Reserve policies effects supply and demand and, therefore, stock prices.

Appendices

market_simulation.R

```
#####
##      LIBRARIES      ##
#####

library("ggplot2")
library("gridExtra")
library("rmutil")
library("rvest")
library("lemon")

#####
##      CONSTANTS      ##
#####

TICK_SIZE <- 0.001
TICKERS_V <- cbind("GS", "BAC", "SWKS", "GOOS", "UPS", "JPM")
NUM_STOCKS <- length(TICKERS_V)

#####
##      FUNCTIONS      ##
#####

#####
## BRIEF: Gets actual price of a stock from the Internet.      ##
## PARAM ticker: Stock exchange code for stock of interest      ##
##      (e.g., BAC is the ticker for Bank of America Corporation). ##
## RETURNS: Current stock price as a numeric type.      ##
#####

get_current_price <- function(ticker)
{
  ## Download HTML data from stock's Yahoo finance page
  url <- paste("https://finance.yahoo.com/quote/", ticker, "?p=", ticker, sep = "")
  html_data <- read_html(url)

  ## Extract price data from Yahoo finance page
  div_span_elems <- html_data %>%
    html_nodes('div span') %>%
```

```

        html_text()
current_price <- as.numeric(div_span_elems[11])

## Indicate that price data for current stock has been extracted
print(paste("--> ", ticker, " price download complete: ", "$", div_span_elems[11], " per
    share", sep = ""))

## Return current price to caller
return(invisible(current_price))
}

#####
## BRIEF: Distributes money supply in economy according to Pareto    ##
## distribution.                                                    ##
## PARAM step_size: Size of movements along Pareto CDF.            ##
## PARAM supply: Total amount of money to distribute amongst the traders. ##
## PARAM n_traders: Number of traders to distribute money to.      ##
## PARAM loc: Location parameter of Pareto distribution.           ##
## PARAM disp: Dispersion parameter of Pareto distribution.        ##
## RETURNS: Vector containing the amount of money each trader has been ##
## allotted.                                                         ##
#####
distribute_wealth_pareto <- function(step_size = 0.1, supply = 1e12, n_traders = 500, loc =
    1, disp = 1.5) {
    ## Allocate memory for vector
    wealth_v <- rep(NA, n_traders)

    wealth_v[1] <- ppareto(step_size, loc, disp) * supply
    wealth_v[2 : n_traders] <- supply * (ppareto(seq(from = 2 * step_size, by = step_size,
        length = n_traders - 1), loc, disp) -
        ppareto(seq(from = step_size, by = step_size, length = n_traders - 1), loc,
            disp))

    return(invisible(wealth_v))
}

#####
## BRIEF: Distributes shares of stock according to Pareto distribution. ##
## PARAM step_size: Size of movements along Pareto CDF.            ##

```

```

## PARAM stock_names_v: Vector of stock tickers.      ##
## PARAM n_shares_v: Number of shares outstanding for each stock. ##
## PARAM n_traders: Number of traders to distribute the shares to. ##
## PARAM loc: Location parameter of Pareto distribution. ##
## PARAM disp: Dispersion parameter of Pareto distribution. ##
## RETURNS: Data-frame containing share ownership information. ##
#####
distribute_shares_pareto <- function(step_size = 0.1, stock_names_v, n_shares_v, n_traders =
  500, loc = 1, disp = 3)
{
  ## Allocate memory for data-frame
  ownership_df <- data.frame(matrix(NA, nrow = n_traders, ncol = length(stock_names_v)))
  colnames(ownership_df) <- stock_names_v

  for (k in 1 : length(stock_names_v)) {
    ownership_v <- rep(NA, n_traders)
    ownership_v[1] <- ppareto(step_size, loc, disp) * n_shares_v[k]
    ownership_v[2 : n_traders] <- n_shares_v[k] * (ppareto(seq(from = 2 * step_size, by =
      step_size, length = n_traders - 1), loc, disp) -
      ppareto(seq(from = step_size, by = step_size, length = n_traders -
        1), loc, disp))
    ownership_df[stock_names_v[k]] <- ownership_v
  }

  return(invisible(ownership_df))
}

#####
## BRIEF: Displays historical stock charts after completion of simulation. ##
## PARAM stocks_df: Dataframe of historical stock price data. ##
## RETURNS: Void. ##
#####
display_stock_charts <- function(stocks_df)
{
  n_stocks <- ncol(stocks_df) - 1
  q_step <- floor(nrow(stocks_df) / 4)
  q_x <- seq(from = q_step, by = q_step, length = 3)
  col_det <- apply(stocks_df[, 1 : n_stocks], MARGIN = 2, FUN = function(x_v)
    x_v[length(x_v)] - x_v[1])
  colors <- ifelse(col_det < 0, "#FF0000", "#119334")

```

```

tickers_l <- as.list(colnames(stocks_df[ , 1 : n_stocks]))

plots <- mapply(tickers_l, colors, FUN = function(u, v)
  ggplot(stocks_df, aes(x = Time)) +
  geom_line(aes_string(y = u), colour = v),
  #geom_point(aes(x = 2, y = 10), color = "black", fill = "blue", alpha = 0.4, size =
    3, stroke = 1.5, shape = 21),
  SIMPLIFY = FALSE)

grid.arrange(grobs = plots, nrow = length(colnames(stocks_df)) / 2)
}

```

```

#####
## BRIEF: Runs simulation of the stock market.          ##
## PARAM n_steps: Number time steps to run the simulation for. ##
## PARAM n_hft_traders: Number of traders using a high-frequency ##
##                  trading strategy.                  ##
## PARAM n_fund_traders: Number traders who use fundamental analysis ##
##                  to make trading decisions.          ##
## PARAM n_mom_traders: Number of traders who make trading decisions ##
##                  based on upward and downward price trends. ##
## PARAM n_rand_traders: Number of traders who make trading decisions ##
##                  randomly.                            ##
## RETURNS: Data-frame containing stock-ownership and net-worth data ##
##          about each trader in the simulation.        ##
#####
market_sim <- function(n_steps = 400,
  n_hft_traders = 1000, hft_freq = 10,
  n_fund_traders = 1000,
  n_mom_traders = 1000, mavg_l = 40,
  n_rand_traders = 1000)
{
  ## Times at which quarterly reports will be issued by the companies
  Q1 <- floor(n_steps / 4)
  Q2 <- floor(2 * (n_steps / 4))
  Q3 <- floor(3 * (n_steps / 4))

  #####
  ##          Set-up stocks          ##

```

```

##          and          ##
##          companies      ##
#####

## Create a dataframe to store historical stock price data
stocks_df <- data.frame(matrix(NA, nrow = n_steps, ncol = length(TICKERS_V)))
colnames(stocks_df) <- TICKERS_V

## Download stock prices from Yahoo! finance.
print("Downloading current stock prices ...")
stocks_df[1, ] <- apply(TICKERS_V, MARGIN = 2, FUN = function(t) get_current_price(t))
print("All downloads complete. Running simulation ...")

## Set financial health of each company
gs_health <- list(p_vneg = 0.1, p_neg = 0.3, p_pos = 0.4, p_vpos = 0.2)
bac_health <- list(p_vneg = 0.1, p_neg = 0.2, p_pos = 0.4, p_vpos = 0.3)
swks_health <- list(p_vneg = 0.05, p_neg = 0.2, p_pos = 0.5, p_vpos = 0.25)
goos_health <- list(p_vneg = 0.2, p_neg = 0.3, p_pos = 0.4, p_vpos = 0.1)
ups_health <- list(p_vneg = 0.05, p_neg = 0.5, p_pos = 0.3, p_vpos = 0.15)
jpm_health <- list(p_vneg = 0.05, p_neg = 0.2, p_pos = 0.3, p_vpos = 0.45)

comp_healths_l <- list(gs = gs_health, bac = bac_health, swks = swks_health, goos =
  goos_health, ups = ups_health, jpm = jpm_health)

## Number of shares outstanding for each company
gs_shrs_outs <- 4000
bac_shrs_outs <- 4000
swks_shrs_outs <- 4000
goos_shrs_outs <- 4000
ups_shrs_outs <- 4000
jpm_shrs_outs <- 4000

shrs_outs_v <- cbind(gs_shrs_outs, bac_shrs_outs, swks_shrs_outs, goos_shrs_outs,
  ups_shrs_outs, jpm_shrs_outs)

#####
##          Set-up traders          ##
#####

n_traders <- n_hft_traders + n_fund_traders + n_mom_traders + n_rand_traders
tags_m <- sample(c(rep("HFT", n_hft_traders), rep("FUND", n_fund_traders),
  rep("MOM", n_mom_traders), rep("RAND", n_rand_traders)))

```

```

capital_v <- distribute_wealth_pareto(n_traders = n_traders)
ownership_df <- distribute_shares_pareto(n_traders = n_traders, stock_names_v = TICKERS_V,
  n_shares_v = shrs_outs_v)
traders_df <- data.frame(1 : n_traders, capital_v, tags_m, ownership_df)
colnames(traders_df) <- c("ID", "CAPITAL", "STRATEGY", TICKERS_V)

mom_df <- subset(traders_df, STRATEGY == "MOM")
fund_df <- subset(traders_df, STRATEGY == "FUND")
hft_df <- subset(traders_df, STRATEGY == "HFT")
rand_df <- subset(traders_df, STRATEGY == "RAND")

#####
##          Run market simulation      ##
#####
# q_results_l <- vector("list", length(TICKERS_V))
q_results_l <- lapply(comp_healths_l, FUN = function(x) issue_earnings_report(p_vneg =
  x$p_vneg, p_neg = x$p_neg,
                                p_pos = x$p_pos, p_vpos = x$p_vpos))

## Sum used to compute price's moving average
sum_last_obs_v <- rep(0, NUM_STOCKS)
sum_last_obs_v <- as.numeric(stocks_df[1,])

demand_v <- rep(0, NUM_STOCKS)
supply_v <- rep(0, NUM_STOCKS)

for (t in 1 : n_steps) {

  current_prices_v <- as.numeric(stocks_df[t, ])

  #####
  ## Execute trading strategies ##
  #####

  #####
  ## Random supply and demand needed to set market in motion ##
  #####

  supply_v <- apply(shrs_outs_v, MARGIN = 2, FUN = function(x) runif(n = 1, min = 0, max
    = x))
  demand_v <- runif(n = NUM_STOCKS, min = 0, max = n_traders)

```

```

#####
## MOMENTUM ##
#####

## Calculate n-step moving average for each stock
sum_last_obs_v <- ifelse(rep(t, NUM_STOCKS) <= mavg_l, sum_last_obs_v +
  as.numeric(stocks_df[t, ]),
  sum_last_obs_v + as.numeric(stocks_df[t, ]) - as.numeric(stocks_df[t -
    mavg_l, ]))

mavg_v <- sum_last_obs_v / mavg_l

## If price exceeds n-step moving average by 2%, buy 100 shares
## else if price is below n-step moving average by 2%, sell 100 shares
## else hold
if (t %% 40 == 0) { ## Trade every 40 steps
  for (j in 1 : NUM_STOCKS) {
    if (current_prices_v[j] >= mavg_v[j] * 1.02) {

      ## MORE LIKELY TO BUY STOCKS
      for (k in 1 : n_mom_traders) {

        if (mom_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {

          decision <- sample(c("BUY", "SELL", "HOLD"), 1, prob = c(0.6, 0.3, 0.1))

          if (decision == "BUY") {

            demand_v[j] <- demand_v[j] + 10
            supply_v[j] <- supply_v[j] - 10
            mom_df["CAPITAL"][k,] <- mom_df["CAPITAL"][k,] - 100 *
              current_prices_v[j]
            mom_df[TICKERS_V[j]][k,] <- mom_df[TICKERS_V[j]][k,] + 100

          } else if (decision == "SELL") {

            demand_v[j] <- demand_v[j] - 10
            supply_v[j] <- supply_v[j] + 10
            mom_df["CAPITAL"][k,] <- mom_df["CAPITAL"][k,] + 100 *
              current_prices_v[j]

```

```

        mom_df[TICKERS_V[j]][k,] <- mom_df[TICKERS_V[j]][k,] - 100

    }

}

} else if (current_prices_v[j] <= mavg_v[j] * 0.98) {

    ## MORE LIKELY TO SELL STOCKS
    for (k in 1 : n_mom_traders) {

        if (mom_df[TICKERS_V[j]][k,] >= 100) {

            decision <- sample(c("BUY", "SELL", "HOLD"), 1, prob = c(0.3, 0.6, 0.1))

            if (decision == "SELL") {

                demand_v[j] <- demand_v[j] - 10
                supply_v[j] <- supply_v[j] + 10
                mom_df["CAPITAL"][k,] <- mom_df["CAPITAL"][k,] + 100 *
                    current_prices_v[j]
                mom_df[TICKERS_V[j]][k,] <- mom_df[TICKERS_V[j]][k,] - 100

            } else if (decision == "BUY") {

                demand_v[j] <- demand_v[j] + 10
                supply_v[j] <- supply_v[j] - 10
                mom_df["CAPITAL"][k,] <- mom_df["CAPITAL"][k,] - 100 *
                    current_prices_v[j]
                mom_df[TICKERS_V[j]][k,] <- mom_df[TICKERS_V[j]][k,] + 100

            }

        }

    }

}

}

}

#####
## HFT ##
#####

```

```

## If last quarterly report was very positive, buy stock whenever price is 1% below
    moving average
##
    sell stock whenever price is 2% above moving average

## If last quarterly report was positive, buy stock whenever price is 2% below moving
    average
##
    sell stock whenever price is 2% above moving average

## If last quarterly report was negative, buy stock whenever price is 3% below moving
    average
##
    sell stock whenever price is 1% above moving average

## If last quarterly report was very negative, buy stock whenever price is 4% below
    moving average
##
    sell stock whenever price is 1% above moving average

if (t %% hft_freq == 0) { ## Trade every hft_freq steps

  for (j in 1 : NUM_STOCKS) {
    if (q_results_1[[j]] == "VGOOD") {
      if (current_prices_v[j] <= mavg_v[j] * 0.99) { ## BUY
        for (k in 1 : n_hft_traders) {
          if (hft_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {

            demand_v[j] <- demand_v[j] + 10
            supply_v[j] <- supply_v[j] - 10
            hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] - 100 *
              current_prices_v[j]
            hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] + 100

          }
        }
      } else if (current_prices_v[j] >= mavg_v[j] * 1.02) { ## SELL
        for (k in 1 : n_hft_traders) {
          if (hft_df[TICKERS_V[j]][k,] >= 100) {
            demand_v[j] <- demand_v[j] - 10
            supply_v[j] <- supply_v[j] + 10
            hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] + 100 *
              current_prices_v[j]
            hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] - 100
          }
        }
      }
    }
  }
}

```

```

    }
  }
}
} else if (q_results_1[[j]] == "GOOD") {
  if (current_prices_v[j] <= mavg_v[j] * 0.98) { ## BUY
    for (k in 1 : n_hft_traders) {
      if (hft_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {

        demand_v[j] <- demand_v[j] + 10
        supply_v[j] <- supply_v[j] - 10
        hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] - 100 *
          current_prices_v[j]
        hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] + 100

      }
    }
  }
} else if (current_prices_v[j] >= mavg_v[j] * 1.02) { ## SELL
  for (k in 1 : n_hft_traders) {
    if (hft_df[TICKERS_V[j]][k,] >= 100) {
      demand_v[j] <- demand_v[j] - 10
      supply_v[j] <- supply_v[j] + 10
      hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] + 100 *
        current_prices_v[j]
      hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] - 100
    }
  }
}
}

} else if (q_results_1[[j]] == "BAD") {
  if (current_prices_v[j] <= mavg_v[j] * 0.97) { ## BUY
    for (k in 1 : n_hft_traders) {
      if (hft_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {

        demand_v[j] <- demand_v[j] + 10
        supply_v[j] <- supply_v[j] - 10
        hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] - 100 *
          current_prices_v[j]
        hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] + 100

      }
    }
  }
}
}

```

```

    } else if (current_prices_v[j] >= mavg_v[j] * 1.01) { ## SELL
      for (k in 1 : n_hft_traders) {
        if (hft_df[TICKERS_V[j]][k,] >= 100) {
          demand_v[j] <- demand_v[j] - 10
          supply_v[j] <- supply_v[j] + 10
          hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] + 100 *
            current_prices_v[j]
          hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] - 100
        }
      }
    }

  } else {
    if (current_prices_v[j] <= mavg_v[j] * 0.96) { ## BUY
      for (k in 1 : n_hft_traders) {
        if (hft_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {

          demand_v[j] <- demand_v[j] + 10
          supply_v[j] <- supply_v[j] - 10
          hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] - 100 *
            current_prices_v[j]
          hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] + 100

        }
      }
    } else if (current_prices_v[j] >= mavg_v[j] * 1.01) { ## SELL
      for (k in 1 : n_hft_traders) {
        if (hft_df[TICKERS_V[j]][k,] >= 100) {
          demand_v[j] <- demand_v[j] - 10
          supply_v[j] <- supply_v[j] + 10
          hft_df["CAPITAL"][k,] <- hft_df["CAPITAL"][k,] + 100 *
            current_prices_v[j]
          hft_df[TICKERS_V[j]][k,] <- hft_df[TICKERS_V[j]][k,] - 100
        }
      }
    }
  }

}

}

```

```

}

#####
## FUNDAMENTAL ##
#####

## Only buy and sell stocks after quarterly reports have been issued.
## More likely to sell if report was negative
## More likely to buy if report was positive
if (t == Q1 || t == Q2 || t == Q3) {

  q_results_1 <- lapply(comp_healths_1, FUN = function(x) issue_earnings_report(p_vneg
    = x$p_vneg, p_neg = x$p_neg,
                                p_pos = x$p_pos, p_vpos = x$p_vpos))
  ## Fundamental traders make trades based on quarterly results
  for (j in 1 : length(q_results_1)) {
    if (q_results_1[[j]] == "VBAD") {

      prob_buy <- 0.05
      prob_sell <- 0.9
      prob_hold <- 0.05

    } else if (q_results_1[[j]] == "BAD") {
      prob_buy <- 0.1
      prob_sell <- 0.8
      prob_hold <- 0.1

    } else if (q_results_1[[j]] == "GOOD") {

      prob_buy <- 0.8
      prob_sell <- 0.1
      prob_hold <- 0.1

    } else {

      prob_buy <- 0.9
      prob_sell <- 0.05
      prob_hold <- 0.05

    }
  }
}

```

```

for (k in 1 : n_fund_traders) {
  decision <- sample(c("BUY", "SELL", "HOLD"), 1, prob = c(prob_buy, prob_sell,
    prob_hold))

  if (decision == "BUY") {
    if (fund_df["CAPITAL"][k,] >= 100 * current_prices_v[j]) {
      demand_v[j] <- demand_v[j] + 10
      supply_v[j] <- supply_v[j] - 10
      fund_df["CAPITAL"][k,] <- fund_df["CAPITAL"][k,] - 100 *
        current_prices_v[j]
      fund_df[TICKERS_V[j]][k,] <- fund_df[TICKERS_V[j]][k,] + 100
    }
  } else if (decision == "SELL") {
    if (fund_df[TICKERS_V[j]][k,] >= 100) {
      demand_v[j] <- demand_v[j] - 10
      supply_v[j] <- supply_v[j] + 10
      fund_df["CAPITAL"][k,] <- fund_df["CAPITAL"][k,] + 100 *
        current_prices_v[j]
      fund_df[TICKERS_V[j]][k,] <- fund_df[TICKERS_V[j]][k,] - 100
    }
  }
}

#####
## Calculate new stock prices based on supply and demand ##
#####
demsupp_diff_v <- demand_v - supply_v
stocks_df[t + 1, ] <- mapply(stocks_df[t, ], demsupp_diff_v, FUN = function(x, y)
  ifelse(x <= 0, 0, x + TICK_SIZE * y + rnorm(1, mean = 0, sd = 0.002)))

}

stocks_df["Time"] <- 0 : (n_steps)
display_stock_charts(stocks_df)

return(invisible(traders_df))
}

```

```
issue_earnings_report <- function(p_vneg = 0.25, p_neg = 0.25, p_pos = 0.25, p_vpos = 0.25)
{
  result_lvls_v <- c("VBAD", "BAD", "GOOD", "VGOOD")
  probs_v <- c(p_vneg, p_neg, p_pos, p_vpos)
  result <- sample(result_lvls_v, size = 1, prob = probs_v)

  return(invisible(result))
}
```
